HDOC Day 30

AIM

To write and execute a menu-driven C program to perform bubble sorting and binary searching, insertion and deletion at valid positions, and right rotation of an integer array by k positions.

THEORY

Bubble Sort: Compares adjacent elements and swaps them if they are in the wrong order. **Binary Search:** Finds a key in a sorted array by repeatedly halving the search interval. **Insertion:** Shifts elements right to make space at a given position. **Deletion:** Shifts elements left to remove an element. **Right Rotation:** Moves each element k positions to the right with wraparound.

ALGORITHM

General steps: 1. Start. 2. Read array size and elements. 3. Display the menu. 4. Read the choice and perform the corresponding operation. 5. Repeat until Exit is selected. 6. Stop.

Option 1 - Bubble Sort and Binary Search: (i) Compare adjacent elements using nested loops and swap if required. (ii) Display sorted array. (iii) Read search key. (iv) Set low = 0 and high = n - 1. (v) While low <= high, compute mid; if key matches, report position; otherwise adjust low or high. (vi) If not found, display a message.

Option 2 - Insertion and Deletion: (i) Ask whether to insert or delete. (ii) For insertion, check capacity and position 1 to n + 1; shift elements one place right, insert value, increase n. (iii) For deletion, check position 1 to n; shift following elements left and decrease n. (iv) Display updated array.

Option 3 - Right Rotation: (i) Read k. (ii) Calculate k = k % n. (iii) Repeat k times: save last element, shift all elements right by one, and place saved element at index 0. (iv) Display rotated array.

C PROGRAM

```
#include <stdio.h>
#define MAX 100

void display(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int a[MAX], n, i, j, t, choice, key;
    int low, high, mid, found, op, pos, val, k;

    printf("Enter number of elements (1-100): ");
    scanf("%d", &n);
    if (n < 1 || n > MAX) return 0;
    printf("Enter %d elements: ", n);
```

```
for (i = 0; i < n; i++) scanf("%d", &a[i]);

do {
    printf("\n1. Bubble Sort and Binary Search\n");
    printf("2. Insertion or Deletion\n");
    printf("3. Right Rotation\n4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
    case 1:
        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) {
                    t = a[j]; a[j] = a[j + 1];
                    a[j + 1] = t;
                }
        printf("Sorted array: "); display(a, n);
        printf("Enter key to search: ");
        scanf("%d", &key);
        low = 0; high = n - 1; found = -1;
        while (low <= high) {
            mid = (low + high) / 2;
            if (a[mid] == key) {
                found = mid; break;
            } else if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        if (found == -1) printf("Element not found\n");
        else printf("Found at position %d\n", found + 1);
        break;

    case 2:
        printf("1. Insert  2. Delete: ");
        scanf("%d", &op);
        if (op == 1) {
            if (n == MAX) {
                printf("Array full\n"); break;
            }
            printf("Enter position and value: ");
            scanf("%d%d", &pos, &val);
            if (pos < 1 || pos > n + 1) {
                printf("Invalid position\n"); break;
            }
            for (i = n; i >= pos; i--) a[i] = a[i-1];
            a[pos - 1] = val; n++;
        } else if (op == 2) {
            printf("Enter position to delete: ");
            scanf("%d", &pos);
            if (pos < 1 || pos > n) {
                printf("Invalid position\n"); break;
            }
            for (i = pos - 1; i < n - 1; i++)
                a[i] = a[i + 1];
            n--;
        } else {
            printf("Invalid operation\n"); break;
        }
        printf("Updated array: "); display(a, n);
        break;

    case 3:
        if (n == 0) {
            printf("Array is empty\n"); break;
        }
        printf("Enter k (non-negative): ");
        scanf("%d", &k);
```

```
            if (k < 0) {
                printf("Invalid k\n"); break;
            }
            k = k % n;
            while (k--) {
                t = a[n - 1];
                for (i = n - 1; i > 0; i--)
                    a[i] = a[i - 1];
                a[0] = t;
            }
            printf("Rotated array: "); display(a, n);
            break;

        case 4: printf("Program ended\n"); break;
        default: printf("Invalid choice\n");
        }
    } while (choice != 4);
    return 0;
}
```

SAMPLE OUTPUT / TEST CASES

The following is one continuous example run. The array is updated after each operation.

```
Enter number of elements (1-100): 5
Enter 5 elements: 5 2 9 1 7

Enter choice: 1
Sorted array: 1 2 5 7 9
Enter key to search: 7
Found at position 4

Enter choice: 2
1. Insert  2. Delete: 1
Enter position and value: 3 4
Updated array: 1 2 4 5 7 9

Enter choice: 2
1. Insert  2. Delete: 2
Enter position to delete: 2
Updated array: 1 4 5 7 9

Enter choice: 3
Enter k (non-negative): 2
Rotated array: 7 9 1 4 5

Enter choice: 4
Program ended
```

RESULT

The menu-driven C program was successfully written and its operations were verified using sample input: Bubble Sort arranged the elements in ascending order; Binary Search located the required element; insertion and deletion updated the array at valid positions; and right rotation shifted the array elements by k positions.